Vetra Framework — Proposta Técnica

Oct 29, 2026 · @Denilson Silva

Sumário executivo

O Vetra é um framework web em Ruby 4, exclusivo para PostgreSQL, que trata a aplicação como um modelo verificável — o Grafo da Aplicação — em vez de um conjunto de classes ligadas por metaprogramação em runtime.

A aposta não é ser um Rails mais organizado. É tornar trivial o que hoje é caro em Rails: telas em tempo real sem código de broadcast, processos de negócio que duram dias, isolamento multiempresa verificado por testes adversariais, ferramentas de conformidade com a LGPD e operações que agentes de IA executam com identidade delegada e permissões limitadas.

A adoção começa dentro de aplicações Rails existentes, um domínio por vez. O Vetra só se torna independente depois de provar vantagem medida em um módulo comercial real.

A fundação são quatro componentes, validados antes de qualquer outro: Application Graph, motor de comandos, motor de autorização e Vetra Data (Sequel + PostgreSQL com RLS). Consultas reativas são o principal diferencial e entram assim que essa base estiver correta. Ruby::Box, Ractors, workflows duráveis e MCP são extensões opcionais: o núcleo funciona integralmente sem eles.

Contexto

O Ruby 4 trouxe três recursos que um framework novo pode aproveitar como extensões, sem depender deles para ser correto, e o Rails ainda não foi desenhado em torno deles.

Recurso do Ruby 4	Estado	Uso no Vetra
Ruby::Box	Experimental, ativado por RUBY_BOX=1	Opcional: separar definições por domínio, recarga e canário em modo sombra
Ractor::Port e Ractor.shareable_proc	Estável na API, Ractor ainda experimental	Pools de CPU para relatórios, PDFs e serialização
ZJIT	Experimental; ainda abaixo do YJIT	Apenas medição; produção usa YJIT

Versão de referência: Ruby 4.0.9 (23/10/2026). O Ruby::Box isola monkey patches, variáveis globais e de classe, definições e bibliotecas carregadas; entre os usos previstos pelo core estão blue-green deployment dentro do processo e uma futura API de pacotes (Ruby 4.0.0).

O Rails 8.1 é maduro e forte: Solid Queue, Solid Cache e Solid Cable, continuações de jobs, eventos estruturados e Kamal. Suas lacunas reais para sistemas comerciais grandes são outras:

- **Arquitetura implícita.** Regras espalhadas entre models, callbacks, concerns e services; nada verifica a estrutura.
- **Tempo real manual.** O desenvolvedor decide o que transmitir e quando; telas ficam desatualizadas por omissão.
- **Processos longos.** Continuações retomam um job interrompido, mas não orquestram fluxos de dias com timers, sinais e compensações.
- **Multiempresa por disciplina.** O isolamento depende de lembrar o escopo em cada consulta.
- **Dados pessoais sem modelo.** LGPD é tratada caso a caso.
- **IA como cliente externo.** Não há forma nativa de um agente operar o sistema sob as mesmas políticas.

Tese e princípios

Tese: o Rails otimizou a escrita de código no ciclo requisição → resposta; o Vetra otimiza a vida de um sistema comercial em 10 anos, com dados em tempo real, processos longos e código escrito em parte por IA.

1. **A aplicação é um modelo.** Toda declaração é compilada em um grafo verificável. Inconsistências declaradas são erros de boot; o que o Ruby dinâmico não permite analisar vira aviso ou exige declaração explícita.
2. **Brevidade do Rails, sem mágica em runtime.** O DSL é tão curto quanto o do Rails, mas é avaliado uma vez e congelado.
3. **Segurança por padrão, cerimônia por exceção.** O desenvolvedor só escreve o que difere do caso comum; os mecanismos de segurança são inferidos ou gerados, nunca exigidos à mão no caso simples.
4. **Nenhuma autoridade ambiente.** O código só acessa o que declarou: dados no escopo do ator, hosts externos, serviços. O alcance real dessa garantia é documentado (ver Pilar 6).
5. **Uma operação, várias superfícies.** A regra de negócio é escrita uma vez; HTTP, CLI, job e webhook são adaptadores. MCP é opt-in, com controles próprios.

6. **O banco é aliado, não detalhe.** PostgreSQL-only: RLS, replicação lógica, SKIP LOCKED e transações são a fundação.
7. **Garantias verificáveis, não presumidas.** Toda promessa de isolamento, durabilidade ou autorização tem um mecanismo concreto, um nível de garantia declarado e testes reproduzíveis. Testes reduzem risco; não constituem prova formal.
8. **Recursos experimentais nunca definem a confiabilidade do núcleo.** O Vetra é correto sem Ruby::Box, Ractors ou ZJIT; eles só acrescentam.
9. **Produtividade medida no ciclo completo.** Criar, modificar, testar, auditar e manter uma funcionalidade importa mais que economizar linhas.

Posicionamento: **plataforma por dentro, framework por fora**. O runtime de operações, capacidades e efeitos é uma plataforma; a experiência do desenvolvedor é a de um framework. O Vetra não se apresenta como plataforma, porque isso sugere infraestrutura a operar e dificulta a adoção.

Simplicidade e camadas de adoção

Cada mecanismo de segurança acrescentado nas revisões (contexto assinado, Effect Boundary, níveis de garantia) empurra o Vetra para a complexidade; por isso a simplicidade é requisito com o mesmo peso da segurança, protegido por métricas e gate.

O que o desenvolvedor escreve	O que o Vetra infere ou gera
Entidade com `tenant :company`	Escopo do tenant, política RLS, FKs compostas, contexto assinado
Nada além da entidade	Comandos e consultas CRUD, contratos de entrada, rotas, OpenAPI e tipos
Papel com acesso CRUD	Política padrão por entidade
`uses :leads` em um comando	Capacidades e Effect Boundary do comando
Nada	Transação, outbox, auditoria e máscara de dados pessoais

Camadas. Cada camada é opt-in; ninguém precisa conhecer a seguinte para ser produtivo na anterior.

Camada	O desenvolvedor aprende	Para
1. CRUD	Entidade e papéis	Cadastros e telas administrativas

Camada	O desenvolvedor aprende	Para
2. Regras	Comando, política com escopo, evento	Regras de negócio e integrações
3. Tempo real e processos	Consulta reativa, workflow	Dashboards, filas de atendimento, fluxos de dias
4. Avançado	MCP, execução sombra, Ruby::Box, Ractors	Agentes, deploy sofisticado, desempenho

Níveis de garantia, contexto assinado e Effect Boundary são vocabulário do framework e da documentação, não do código da aplicação: aparecem em `vetra explain`, nos erros e nos relatórios.

Práticas que protegem a simplicidade:

- **Erros que ensinam:** quando o grafo recusa algo, a mensagem diz o que falta, por quê e a declaração exata a escrever.
- **Orçamento de conceitos:** um CRUD multiempresa exige no máximo três conceitos (entidade, papel e, opcionalmente, comando); todo conceito novo na camada 1 precisa de ADR.
- **Sem cerimônia corporativa:** nada de injeção de dependência explícita, interfaces, fábricas ou arquivos de configuração para o caso comum.
- **Gerado, não escrito:** código gerado pode ser substituído por código escrito ("ejetar" um comando) sem reescrever o resto.

Exemplo: valores monetários

O desenvolvedor declara apenas `money :valor`; moeda, precisão, arredondamento, formato e interpretação da entrada vêm do tenant e do idioma de quem vê a tela. O valor (armazenamento e cálculo) nunca carrega formato; a apresentação só existe na borda.

Aspecto	Brasil (pt-BR)	Estados Unidos (en-US)
Exibição	R$ 1.234,56	$1,234.56
Separador decimal	vírgula	ponto
Separador de milhar	ponto	vírgula
Símbolo	antes, com espaço	antes, sem espaço
Entrada "1.234"	mil duzentos e trinta e quatro	um vírgula duzentos e trinta e quatro

Aspecto	Brasil (pt-BR)	Estados Unidos (en-US)
API e banco (canônico)	"1234.56" + "BRL"	"1234.56" + "USD"

- **Valor e moeda sempre juntos:** `numeric` + código ISO 4217; moeda padrão do tenant.
- **Nunca ponto flutuante:** decimal exato; Float é recusado pelo grafo em campos monetários.
- **Moedas não se misturam:** somar BRL com USD é erro; conversão explícita, com taxa e data registradas.
- **Moeda ≠ idioma:** um americano vendo reais vê "R$1,234.56"; um brasileiro vendo dólares vê "US$ 1.234,56".
- **Entrada conforme o idioma:** formulários interpretam pelo idioma; APIs só aceitam o formato canônico, com valor como string (evita perda de precisão em JavaScript).
- **Arredondamento em pontos definidos:** padrão comercial (meio para cima), configurável por tenant; arredonda ao persistir e exibir.
- **Rateio exato:** R$ 100,00 em 3 parcelas gera 33,34 + 33,33 + 33,33.
- **Precisão por uso:** transações usam as casas da moeda; preços unitários podem declarar mais casas.
- **Testes:** a formatação brasileira usa espaço não separável entre "R$" e o número; comparações com espaço comum falham.

O Rails já resolve boa parte disso com as gems money e money-rails; a vantagem do Vetra é a integração por padrão — contrato, tipos TypeScript, validação no grafo e idioma por requisição, sem configuração.

Escopo e não-objetivos

O Vetra mira aplicações comerciais multiempresa — CRM, ERP, atendimento, gestão financeira — onde regras, permissões e auditoria dominam o custo.

No escopo

- Aplicações SaaS multiempresa com hierarquias (grupo → empresa → unidade → usuário).
- APIs, interfaces renderizadas no servidor e SPAs em Vue ou React com cliente tipado.
- Processos de negócio longos, integrações por webhook e automações agendadas.
- Operação por agentes de IA sob as mesmas políticas dos usuários.

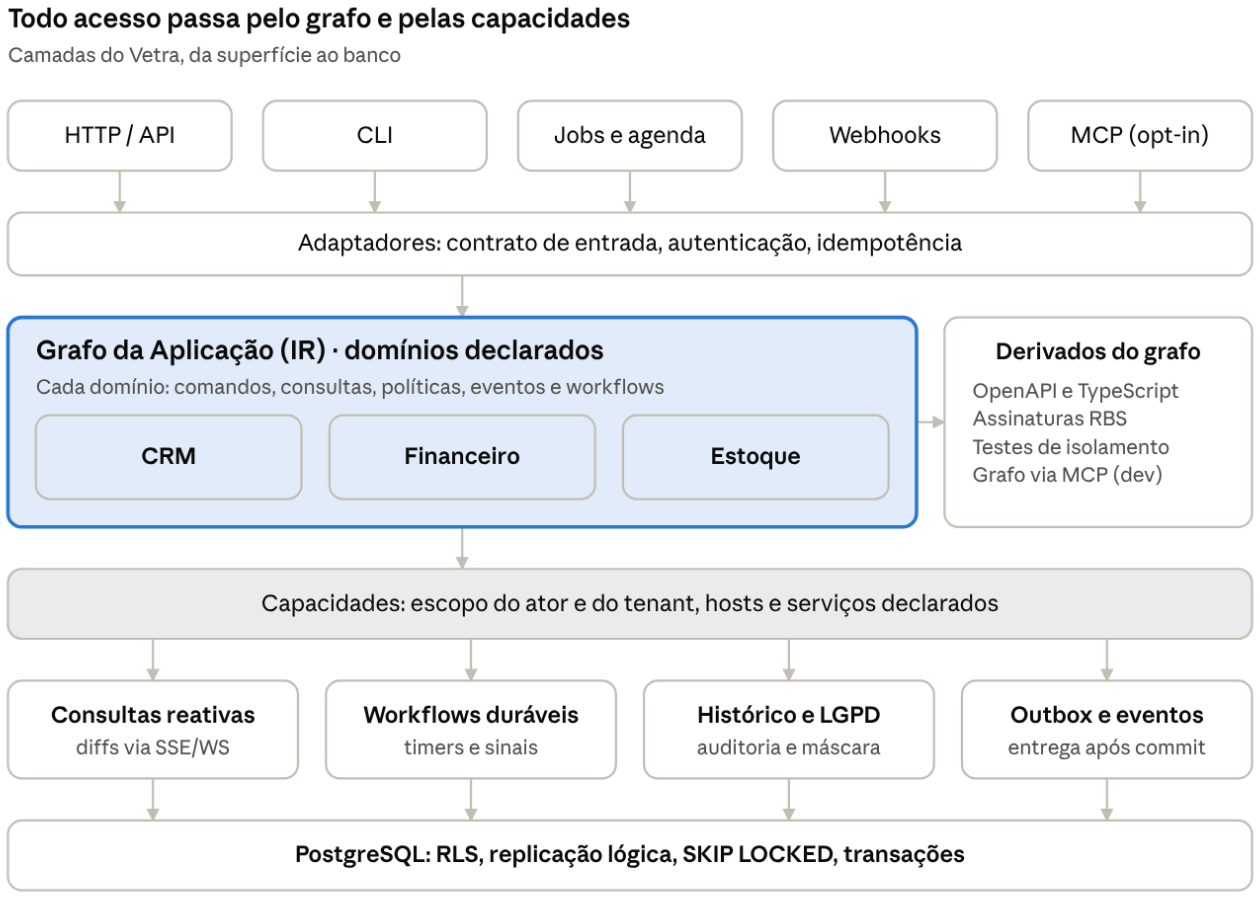
Fora do escopo (decisões deliberadas)

- MySQL, SQLite ou outros bancos no núcleo.

- ORM próprio: a persistência usa Sequel por baixo.
- Servidor HTTP próprio: Falcon como padrão, Puma como alternativa.
- Framework de frontend próprio: apenas SDK tipado e componentes de sincronização.
- Sites de conteúdo, blogs e e-commerce simples, onde o Rails já é a melhor escolha.
- Compatibilidade de API com o Rails: o Vetra convive com ele, não o imita.

Arquitetura

O Vetra tem seis camadas — superfícies, adaptadores, Grafo da Aplicação, capacidades, serviços de runtime e PostgreSQL — e nenhuma camada pula a seguinte.

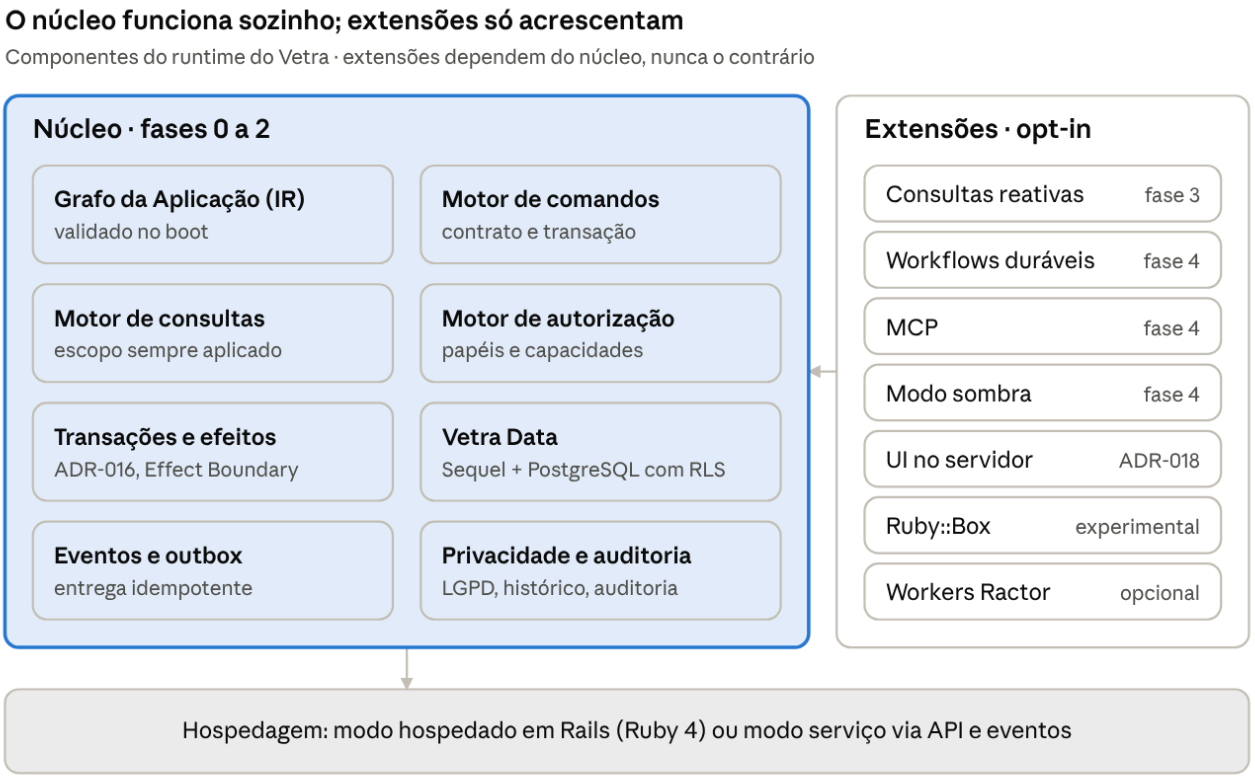


arquitetura do Vetra · 6 camadas

Toda superfície chama o mesmo comando; nenhum código alcança o banco sem passar pelas capacidades e pelo RLS. O grafo também alimenta os artefatos derivados, que nunca são escritos à mão.

Núcleo e extensões

O núcleo são oito componentes entregues nas fases 0 a 2; tudo o que é experimental, caro ou ainda sem ADR fica fora dele, como extensão opt-in.

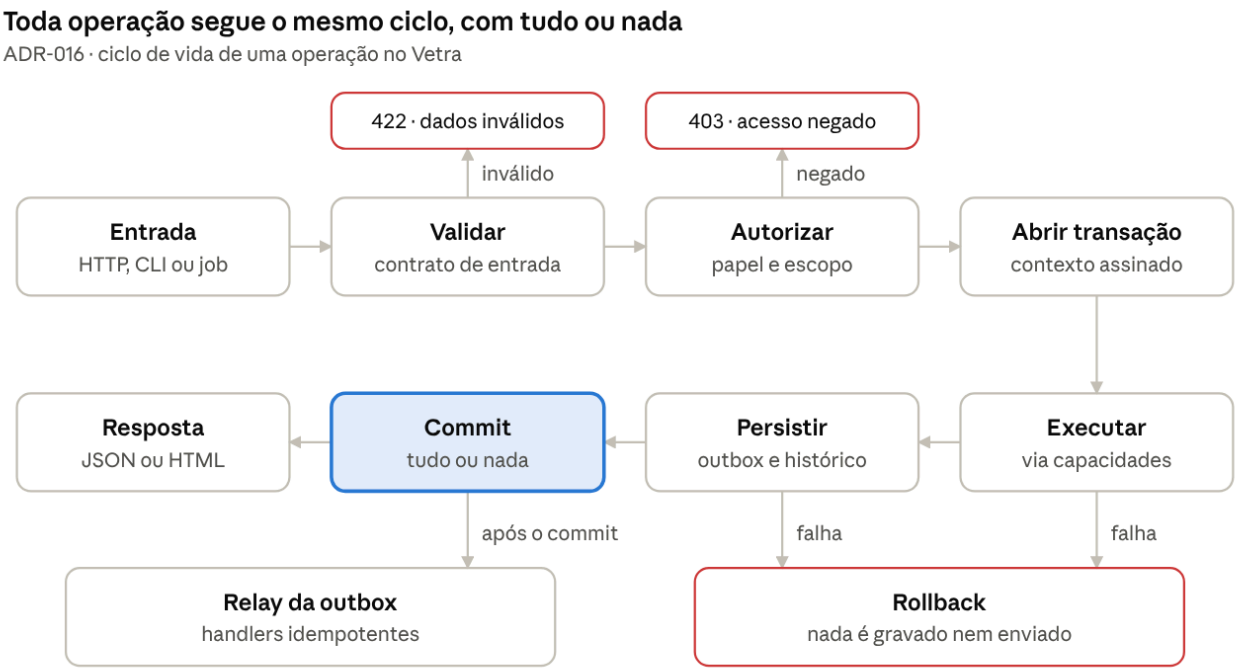


componentes do runtime · núcleo, extensões e hospedagem

Uma extensão pode usar qualquer componente do núcleo; o núcleo nunca depende de uma extensão. A UI no servidor é uma extensão especificada no ADR-018, com Turbo como cliente no navegador; os sinais efêmeros acompanham as consultas reativas (ADR-019).

Ciclo de vida de uma operação

Toda operação, venha de HTTP, CLI, job ou webhook, passa pelas mesmas oito etapas do ADR-016; dados, outbox e histórico são gravados juntos ou não são gravados.



ciclo de vida de uma operação · 8 etapas, 3 saídas de falha

Entrada inválida e acesso negado param antes de abrir a transação. Qualquer falha depois disso desfaz tudo. Os eventos só saem para os handlers depois do commit, pelo relay da outbox.

Estrutura de um projeto

O Rails organiza o código por tipo técnico (`models`, `controllers`, `views`); o Vetra organiza por domínio de negócio, e o tipo de cada arquivo vem da declaração que ele contém (`entity`, `roles`, `query`, `command`, `handler`), não da pasta.

```
meu_app/
├── .vetra/                          # gerado, não editar
│   ├── cache/                       # IR compilada para boot rápido
│   └── graph.json                   # IR versionada do grafo (vai para o Git)
├── app/
│   ├── domains/                     # escrito: um diretório por domínio
│   │   ├── crm/
│   │   │   ├── lead.rb              # entity
│   │   │   ├── queries.rb           # query
│   │   │   ├── roles.rb             # roles
│   │   │   ├── notify_seller.rb     # handler
│   │   │   └── win_lead.rb          # command
│   │   └── financeiro/
│   └── ui/                          # só com --ui=server (ADR-018)
│       ├── assets/                  # CSS e imagens
│       ├── layouts/
│       ├── templates/               # escrito: ERB das páginas
```

```
│       └── pages.rb                # escrito: rotas de página
├── bin/
│   ├── setup                       # prepara o ambiente
│   └── vetra                       # CLI: dev, explain, db:plan, db:migrate
├── config/
│   ├── environments/               # development.rb, production.rb, test.rb
│   ├── locales/                    # traduções (pt-BR.yml)
│   ├── app.rb                      # tenant, moeda, idioma, auth, ui
│   ├── database.rb                 # papéis do banco: aplicação e migração
│   ├── secrets.enc                 # segredos cifrados, inclusive a chave do
contexto assinado
│   └── server.rb                   # servidor Falcon
├── db/
│   ├── plans/                      # gerado: planos de migração
expand/contract
│   ├── policies/                   # gerado: políticas RLS em SQL revisável
(com tenant)
│   ├── schema.sql                  # gerado: retrato do banco
│   └── seeds.rb                    # escrito: dados iniciais
├── lib/                            # código de apoio sem relação com domínio
├── log/
├── public/                         # arquivos estáticos
├── sdk/
│   └── typescript/                 # gerado: cliente tipado (modo API)
├── storage/                        # uploads locais
├── test/
│   ├── domains/                    # escrito: testes por domínio
│   ├── generated/                  # gerado: autorização e isolamento a
partir do grafo
│   └── test_helper.rb
├── tmp/
├── .gitignore
├── .ruby-version                   # 4.0.x
├── config.ru                       # entrada Rack
├── Dockerfile
├── Gemfile
├── Gemfile.lock
└── README.md
```

Comparado a um projeto Rails 8.1, a raiz é menor: não há `Rakefile`, `script/`, `vendor/`, `.kamal/` nem os três esquemas do Solid (`cable`, `cache`, `queue`), porque filas, cache e tempo real usam as tabelas do próprio Vetra no PostgreSQL. Deploy com Kamal continua possível, mas não é gerado por padrão.

No Rails	No Vetra
Migração + model	Entidade (tabela derivada)
Validações no model	Contrato derivado da entidade
Policy (Pundit) + `before_action`	Papéis e políticas declarativas
Controller + service	Comando (escrita) ou consulta (leitura)
Job + agendamento	Handler de evento via outbox
Rotas	`expose` no comando ou `page` na UI
Views administrativas	Geradas por `crud ui: :admin`

Os nove pilares

Os pilares 1 e 6 (com o motor de comandos do pilar 3) formam a fundação; o pilar 4 é o principal diferencial; os pilares 2, 5, 8 e 9 e a superfície MCP são extensões opcionais.

1. Grafo da Aplicação (núcleo)

- **Problema:** no Rails, a arquitetura só existe implícita, espalhada em metaprogramação; nada a verifica.
- **Proposta:** o DSL Ruby alimenta um registro que gera uma **representação intermediária (IR) versionada** do grafo, separada dos objetos Ruby: entidades, comandos, consultas, políticas, eventos, rotas e dependências. A IR é validada e então compilada para o runtime (HTTP, comandos, repositórios, políticas). Dela derivam OpenAPI, tipos TypeScript, RBS, documentação, testes de isolamento e o diff arquitetural entre versões. É a IR, com tipos controlados, que é compartilhada entre Ractors e exposta a ferramentas de IA — não Procs ou objetos arbitrários.
- **Regras de validação:**

Situação	Comportamento
Contrato, política ou entidade referenciada e não declarada	Erro
Dependência proibida entre domínios declarados	Erro
Evento interno com consumidor obrigatório ausente	Erro
Evento público sem consumidor interno	Permitido

Situação	Comportamento
Campo pessoal sem classificação em domínio que a exige	Erro
Dependência dinâmica não analisável	Aviso, ou exigência de declaração explícita

- **Limite:** Ruby é dinâmico; o grafo não detecta todas as dependências de código arbitrário. As garantias valem para o que foi declarado. O boot fica mais lento e exige cache da IR compilada.

2. Isolamento de domínios em três níveis

- **Problema:** em monólitos grandes, fronteiras entre domínios são convenção, e monkey patches vazam.
- **Proposta:**

Nível	Mecanismo	Finalidade
Arquitetural (padrão)	Grafo + verificação de dependências	Impedir acoplamentos indevidos
Runtime (opcional)	Ruby::Box por domínio	Separar definições e monkey patches; recarga por domínio
Operacional (opcional)	Domínio em processo separado	Isolar falhas, memória e recursos sem adotar microsserviços

- **O que Ruby::Box não é:** não é sandbox de segurança (não impede acesso a rede, arquivos ou ao processo) nem isolamento de falhas.
- **Canário em modo sombra (fase 4):** a versão nova de um domínio recebe cópia do tráfego e executa com capacidades simuladas; só as respostas são comparadas. Requisitos: relógio, UUIDs e timestamps fixados pelo simulador (senão há falsas divergências); escritas num repositório simulado sobre snapshot de leitura, nunca numa transação revertida, que seguraria locks da produção; operações com efeitos fora do Effect Boundary ficam inelegíveis para o modo sombra.
- **Limite:** Ruby::Box é experimental, com limitações conhecidas em extensões nativas. O Vetra funciona integralmente sem ele.

3. Uma operação, várias superfícies

- **Problema:** a mesma regra é reimplementada em controller, job, rake task e integração.

- **Proposta:** um comando declarado pode ser exposto como endpoint HTTP, comando de terminal, job ou handler de webhook. Todas as superfícies passam pelo mesmo contrato, política, transação e auditoria. Cada adaptador explicita sua semântica: timeout, idempotência, streaming.
- **MCP (opt-in, extensão):** nenhum comando é exposto a agentes por padrão. Quando exposto:
  - o agente tem **identidade própria** e atua por **delegação** de um usuário, com permissões que são um subconjunto das dele e prazo de validade;
  - comandos classificados como sensíveis (exclusão, financeiro, alteração de permissões) exigem **aprovação humana** explícita;
  - argumentos vindos do agente são tratados como entrada não confiável (risco de prompt injection), sempre validados pelo contrato;
  - limite de taxa e auditoria completa da decisão do agente.
- **Limite:** permissão de um usuário não implica que um agente deva usá-la sem confirmação; a classificação de sensibilidade é responsabilidade de quem declara o comando.

Campos de transição (ADR-021). Campos e valores que representam uma transição de negócio só podem ser alterados por comandos específicos: `set_by:` para um campo, `guarded:` para um valor de enum. O CRUD gerado e as telas administrativas os omitem, e o grafo recusa um comando genérico que tente alterá-los. Sem isso, um `update` genérico contornaria a regra e o evento do comando.

4. Consultas reativas (núcleo)

- **Problema:** no Hotwire e no Action Cable, o desenvolvedor decide manualmente o que transmitir, e o broadcast não reavalia a permissão de quem já assinou; telas ficam desatualizadas por omissão ou continuam recebendo dados após perder o acesso.
- **Proposta:** consultas são assináveis, com entrega incremental (ADR-020).
- **Revisão de autorização:** toda tabela que define permissão (vínculos, papéis, escopos) tem um trigger que incrementa, na mesma transação, a revisão de autorização do tenant. Até um SQL direto gera a revisão.
- **Modos de entrega:**
  - `strict`, padrão em aplicações multiempresa com dados pessoais: antes de cada lote, o processo lê a revisão do tenant (uma linha). Se ela mudou, revalida as assinaturas do tenant antes de entregar.
  - `balanced`, opção explícita para painéis de baixa sensibilidade: permissões em memória, invalidação por evento e revalidação periódica (5 s como ponto de partida).

  - Se o processo não conseguir confirmar a revisão, suspende a entrega de dados protegidos até reconciliar.
- **Agrupamento em dois níveis:**
  - chave estática: tenant, escopo efetivo, campos visíveis, filtros e parâmetros, idioma e template; usuários de empresas diferentes nunca compartilham grupo;
  - partição por evento: os predicados do template (`can?`) são avaliados em memória para a linha alterada, e o trecho é renderizado uma vez por combinação de resultados.
- **Três estratégias, escolhidas pelo motor:** delta direto (`append` , `replace` , `remove`) para alterações simples; reconsulta parcial quando há dependências; reconsulta completa para agregações, joins que mudam o conjunto e perda de sincronia.
- **Ordem e continuidade:** sequência não é ordem de commit, porque um número pode ser atribuído antes de uma transação concorrente terminar. Cada evento da outbox grava o `xid8` da transação, e o distribuidor só lê eventos abaixo de `pg_snapshot_xmin(pg_current_snapshot())` : todas as transações anteriores a esse ponto já terminaram. O distribuidor mantém checkpoint durável e, sem prova de continuidade, ressincroniza. Replicação lógica (LSN) fica para depois.
- **Consistência no cliente:** a escuta começa antes da leitura do estado inicial; na reconexão, o cliente informa o último checkpoint recebido e o servidor envia o diff ou o estado completo.
- **Garantia de revogação:** no modo `strict` , nenhum lote iniciado depois do commit da revogação entrega dados ao usuário revogado. Um lote já iniciado pode terminar (janela de milissegundos), e dados já entregues não podem ser recolhidos: o trecho `live` é limpo, mas o usuário pode tê-los copiado. Ordem estrita entre revogação e entrega em vários processos exigiria um protocolo de coordenação, fora do escopo inicial.
- **Sinais efêmeros** (ADR-019): presença e "digitando" não vão para o histórico, mas passam pelo mesmo motor de autorização. "Digitando" expira sozinho; presença usa leases por usuário e dispositivo, numa tabela `UNLOGGED` no modo só PostgreSQL. Redis é adaptador opcional, adotado quando o NOTIFY começar a aumentar a latência de commit.
- **Infraestrutura:** uma conexão `LISTEN` dedicada por processo, direta ao PostgreSQL, porque o PgBouncer em modo transação não suporta `LISTEN` . Notificações carregam só IDs e versões, pois o payload do NOTIFY é limitado.
- **Cliente** (ADR-018): Turbo no modo servidor; SDK TypeScript com JSON no modo API.
- **Limite:** a revisão por tenant invalida todas as assinaturas do tenant a cada mudança de permissão (simples e correto; granularidade mais fina só após medir); permissões vindas de fora do banco não disparam trigger; o NOTIFY serializa commits sob escrita intensa. Não entra em produção sem o checklist de aceitação do tempo real.

5. Processos duráveis

- **Problema:** fluxos como "lead sem contato em 48h → escalar ao gerente" viram cron + flags + jobs espalhados.
- **Correção em relação à versão anterior:** a proposta original falava em persistir Fibers no PostgreSQL. Isso é inviável: uma Fiber não pode ser serializada e restaurada com sua pilha após o processo reiniciar.
- **Proposta:** execução durável por **histórico de eventos e replay determinístico**. O workflow é escrito em Ruby linear e roda em Fiber sob um `Fiber::Scheduler` determinístico. Cada efeito (comando, timer, sinal) é registrado no histórico no PostgreSQL. Após falha ou deploy, o código é reexecutado e os resultados já registrados vêm do histórico até o ponto de parada. É o modelo do Temporal Ruby SDK, que usa Fibers com scheduler determinístico.
- **Regras:** sem relógio, aleatoriedade, I/O ou threads diretos no código do workflow (bloqueados por TracePoint); efeitos apenas via comandos idempotentes; versão explícita por workflow, com execuções antigas terminando na versão em que começaram; cancelamento, sinais concorrentes e compensações definidos por contrato.
- **Alternativa mantida:** adaptador para Temporal, quando a operação já tiver essa infraestrutura.
- **Limite:** é extensão da fase final, não parte da fundação.

6. Segurança por capacidades e RLS (núcleo)

- **Problema:** isolamento multiempresa depende de lembrar o escopo em cada consulta.
- **Proposta:** não existe acesso global a tabelas. Cada operação recebe apenas as capacidades declaradas: dados no escopo do ator e do tenant, hosts externos permitidos, serviços. O conjunto de dependências externas de uma operação forma seu **Effect Boundary**: tudo o que sai dele é declarado e pode ser interceptado (simulação, testes, modo sombra).
- **Requisitos de RLS:**
  - papel de aplicação sem superusuário, sem `BYPASSRLS` e sem ser dono das tabelas; migrações usam papel separado; `FORCE ROW LEVEL SECURITY` nas tabelas multiempresa;
  - políticas com `USING` (o que pode ser lido e alterado) e `WITH CHECK` (o que pode ser gravado);
  - **contexto assinado:** a aplicação grava na transação um token assinado (HMAC) com tenant, ator e validade curta; a política chama `(SELECT vetra.current_tenant())`, uma função `SECURITY DEFINER` que valida o token com a chave guardada numa tabela que o papel da aplicação não lê;

  - **validação uma vez por consulta:** o `(SELECT ...)` faz o PostgreSQL executar a função como InitPlan, uma vez por instrução, e não por linha;
  - **o resultado validado nunca vai para uma variável de sessão:** variáveis customizadas podem ser alteradas por qualquer SQL, e um sinalizador "verificado" gravado ali poderia ser forjado; a função valida o token a cada consulta;
  - contexto definido com `SET LOCAL` na transação mais externa — dentro de um savepoint, ele seria desfeito no rollback do savepoint;
  - consultas multiempresa fora de transação são recusadas com erro explícito, não com resultado vazio;
  - **negação por padrão:** sem token válido, nenhuma linha é visível;
  - views com `security_invoker` (PostgreSQL 15+), pois por padrão rodam com privilégios do dono e contornam o RLS;
  - **FKs compostas** (`company_id` , `id`): verificações de integridade referencial ignoram o RLS e revelariam a existência de IDs de outro tenant; a FK composta também garante o `same_tenant` ;
  - testes executados com os mesmos papéis de banco de produção.
- **Níveis de ameaça:**

Ameaça	Proteção	Limite
Erro de programação (filtro esquecido)	RLS + negação por padrão	Depende de contexto configurado corretamente
SQL não confiável (injection, gem)	Contexto assinado validado no banco a cada consulta	Um HMAC por consulta; gestão da chave
Código comprometido no processo	Privilégio mínimo e fronteiras operacionais	Nenhum mecanismo no banco é suficiente

- **Modelo de permissões:** RBAC + regras contextuais por escopo (grupo, empresa, revenda, vínculos múltiplos). Acesso administrativo entre empresas é capacidade explícita e auditada, sem desligar o RLS. Toda tabela de permissão tem trigger que incrementa a revisão de autorização do tenant (pilar 4).
- **Caches:** chaves incluem o escopo de autorização; revogação invalida as entradas afetadas.
- **Fronteira de confiança:** a capacidade é garantia para código que usa o runtime do Vetra. Uma gem que abra conexão própria escapa das capacidades; com contexto assinado, não consegue forjar o tenant. Código comprometido no mesmo processo tem o que a aplicação tem, inclusive tokens válidos. Dependências são classificadas como confiáveis, restritas ou fora da garantia.

- **Limite:** o custo do HMAC por consulta e hierarquias com vínculos múltiplos exigem medição.

7. Privacidade, auditoria e histórico

- **Três módulos separados:**

Módulo	Responsabilidade
Vetra Privacy	Classificação de dados pessoais, mascaramento em logs e erros, retenção, exportação e eliminação a pedido do titular
Vetra Audit	Ator, operação, alteração e rastreabilidade
Vetra History	Versões, consultas "como estava em uma data", reconstrução de estado

- **Escopo jurídico:** o Vetra oferece ferramentas de conformidade, não conformidade automática. Base legal, finalidade e as exceções de conservação do art. 16 continuam decisões do controlador; o pedido de eliminação do titular (art. 18, VI) é atendido pelas ferramentas abaixo.
- **Eliminação x histórico — criptografia por envelope:**
  - dados pessoais no histórico e na auditoria são cifrados com uma chave de dados (DEK) por titular;
  - cada DEK é guardada cifrada por uma chave mestra (KEK) mantida fora do banco, em KMS ou HSM;
  - as DEKs ficam num armazenamento de chaves separado do banco principal, com retenção de backup própria e curta;
  - eliminar é destruir a DEK: os dados cifrados ficam ilegíveis sem quebrar a integridade referencial;
  - leituras não chamam o KMS por linha: DEKs decifradas ficam em cache de memória por pouco tempo.
- **Limite honesto dos backups:** um backup do armazenamento de chaves feito antes da eliminação ainda contém a DEK, e a KEK continua existindo. A eliminação só vale para esses backups quando a retenção deles expira, ou após rotação da KEK com descarte da anterior; esse prazo precisa ser documentado.
- **Pseudonimização sozinha não basta:** dado reidentificável continua sendo pessoal, enquanto o anonimizado sai do escopo da LGPD (arts. 12 e 13, §4º).
- **Fora do banco:** backups, arquivos exportados e integrações têm política própria; o Audit registra o que foi enviado a quem.

- **Volume:** histórico configurado por entidade (completo, por campos ou nenhum), com compactação e retenção.

8. Testes por simulação determinística

- **Proposta:** como os efeitos passam pelo Effect Boundary, o runtime simula relógio, rede e falhas a partir de uma semente reproduzível. Três camadas de teste de isolamento: testes gerados do grafo para cada consulta e comando, **testes de mutação** (remover uma política ou cláusula RLS precisa quebrar algum teste) e casos adversariais escritos à mão.
- **Alcance:** testes reduzem risco; não são prova formal de isolamento. O determinismo vale para efeitos dentro do Effect Boundary; o PostgreSQL real não é simulado, e concorrência no banco exige testes de carga dedicados.
- **Limite:** gems que fazem I/O fora do Effect Boundary escapam da simulação.

9. Runtime concorrente

- **Proposta:** Fibers por padrão para I/O — um processo atende milhares de conexões lentas (SSE, webhooks, streaming de LLM). Ractors alimentados por Ractor::Port para trabalho de CPU. YJIT em produção; ZJIT apenas medido.
- **Limite:** muitas extensões C não são seguras para Ractor; o pool de CPU aceita só código verificado.

Níveis de garantia

Toda garantia do Vetra é classificada por mecanismo, não por funcionalidade, para que ninguém presuma mais do que ela oferece; uma mesma funcionalidade pode combinar níveis.

Nível	Significado	Mecanismo de exemplo
Estrutural	Verificado pelo grafo, só para o que foi declarado	Dependência proibida entre domínios
Transacional	Garantido pelo banco sob as condições configuradas	RLS com contexto assinado; rollback na mesma transação
Verificável por testes	Validado em cenários conhecidos	Testes gerados, de mutação e adversariais de isolamento
Operacional	Depende de infraestrutura e execução	Outbox com consumidores idempotentes
Experimental	Sem garantia de estabilidade	Ruby::Box

Exemplo: o isolamento entre empresas é transacional (RLS) e verificável por testes; rotulá-lo com um só nível perderia informação.

Comparativo Rails 8.1 x Vetra

O Rails vence hoje em maturidade e ecossistema; o Vetra só se justifica se vencer nas seis lacunas listadas no Contexto, com medição.

Dimensão	Rails 8.1 (real)	Vetra (proposta)
Arquitetura	Implícita, montada em runtime	Grafo compilado e verificado no boot
Fronteira entre domínios	Convenção, engines ou Packwerk	Grafo + verificação de dependências; Ruby::Box opcional
Regra de negócio	Models, callbacks, services	Comandos com contrato, política e transação
Tempo real	Broadcast manual (Turbo Streams, Action Cable)	Consultas reativas com diffs
Processos longos	Jobs + continuações	Workflows duráveis com timers e sinais
Multiempresa	Gems ou código próprio	Capacidades + RLS, verificados por testes
LGPD	Manual	Classificação de dados nativa

Dimensão	Rails 8.1 (real)	Vetra (proposta)
Auditoria	Gems	Histórico temporal nativo
Superfícies	Uma implementação por superfície	HTTP, CLI, job e webhook do mesmo comando; MCP opt-in
IA	Cliente externo	Grafo via MCP; agentes com identidade delegada e aprovação
Testes	Ecossistema maduro	Simulação determinística + isolamento gerado
Bancos	Vários	Somente PostgreSQL
Produtividade em CRUD	Excelente	Meta: igual ou melhor, medida
Ecossistema e maturidade	Mais de 20 anos	Inexistente

Quando escolher o Rails: CRUD simples, sites de conteúdo, equipes que dependem de gems com integração Rails, sistemas que precisam de estabilidade imediata. Quando o Vetra valeria a aposta: sistemas multiempresa com permissões complexas, telas operacionais em tempo real e processos longos.

Comparação prática de código

A mesma funcionalidade nos dois frameworks: cadastrar um lead, validar os dados, verificar a permissão do usuário na revenda e registrar um evento durável. O código Rails é idiomático para Rails 8.1 com Pundit; o código Vetra é a sintaxe proposta e não existe ainda.

Ruby on Rails 8.1

```
# app/models/lead.rb
class Lead < ApplicationRecord
  belongs_to :company
  belongs_to :dealer

  enum :status, %w[new contacted won lost].index_by(&:itself),
       default: "new", prefix: true

  validates :name, presence: true, length: { in: 2..160 }
  validates :email, format: { with: URI::MailTo::EMAIL_REGEXP }, allow_blank:
true
```

```
  validate :dealer_belongs_to_company

  private

  def dealer_belongs_to_company
    return if dealer.nil? || dealer.company_id == company_id

    errors.add(:dealer, "não pertence à empresa")
  end
end
```

```
# app/policies/lead_policy.rb (Pundit)
class LeadPolicy < ApplicationPolicy
  def create?
    user.permission?(:create_leads, dealer_id: record.dealer_id)
  end
end
```

```
# app/services/leads/create.rb
module Leads
  class Create
    def self.call(lead:, actor:)
      return false unless lead.valid?

      Lead.transaction do
        lead.save!

        OutboxEvent.create!(
          name: "crm.lead.created",
          company_id: lead.company_id,
          payload: { lead_id: lead.id, actor_id: actor.id }
        )

        AuditLog.create!(actor:, action: "lead.create", record: lead)
      end

      true
    end
  end
end
```

```
# app/controllers/api/v1/leads_controller.rb
module Api
  module V1
    class LeadsController < ApplicationController
      def create
```

```
        lead = Current.company.leads.new(lead_params)
        authorize lead

        if Leads::Create.call(lead:, actor: Current.user)
          render json: LeadSerializer.new(lead), status: :created
        else
          render json: { errors: lead.errors }, status:
:unprocessable_content
        end
      end

      private

      def lead_params
        params.expect(lead: [:name, :email, :dealer_id])
      end
    end
  end
end
```

Não mostrado, mas necessário: models OutboxEvent e AuditLog, job que despacha a outbox, ApplicationPolicy, Current, LeadSerializer, o método permission? do usuário, rota e, para RLS, um SET LOCAL do tenant em cada transação.

Vetra (sintaxe proposta)

```
# app/domains/crm/schema/lead.rb
entity :lead do
  tenant :company
  belongs_to :dealer, same_tenant: true

  string :name,  required: true, length: 2..160
  email  :email, optional: true, personal: true
  enum   :status, %w[new contacted won lost], default: "new"

  history :full
end
```

```
# app/domains/crm/policies/lead.rb
policy :lead do
  allow :create, permission: :create_leads, scope: :dealer
end
```

```
# app/domains/crm/commands/create_lead.rb
command :create_lead do
  input :lead, only: %i[name email dealer_id]
  authorize :create, :lead, scope_from: :dealer_id
  uses :leads

  emits "crm.lead.created", with: %i[id dealer_id]

  call { |input, leads| leads.create!(input) }

  expose http: "POST /api/v1/leads", cli: true
end
```

Implícito pelo framework: tenant obtido do ator, SET LOCAL para RLS, transação, gravação do evento na outbox, histórico com ator e estado anterior, máscara do e-mail em logs, resposta 422 padronizada, OpenAPI e tipos TypeScript.

O que cada um cobre

Responsabilidade	Rails 8.1	Vetra (proposta)
Validação de campos	Model (validates)	Entidade
Revenda da mesma empresa	Validação manual no model	same_tenant: true
Permissão por revenda	Pundit + permission? próprio	Política declarativa com escopo
Isolamento por empresa	Current.company por convenção	Capacidade obrigatória; sem ela não há consulta
RLS no banco	Manual, não mostrado	Automático
Transação	Explícita no service	Implícita no comando
Evento durável	OutboxEvent + job próprios	emits, com outbox nativa
Auditoria	AuditLog manual ou gem	history :full
Dado pessoal (LGPD)	Filtros de log manuais	personal: true
CLI e ferramenta MCP	Implementação separada	expose cli: true; MCP só com bloco próprio de delegação e aprovação
OpenAPI e tipos TypeScript	Gems adicionais	Gerados do grafo

Leitura honesta: o Vetra mostra cerca de 28 linhas contra cerca de 75 no Rails, sem contar o que o Rails ainda precisa fora dos arquivos exibidos. Mas essa brevidade vem de convenções que ainda teriam de ser construídas e testadas. Um app Rails maduro pode chegar perto com gems como Pundit, audited e uma gem de outbox; a vantagem real do Vetra está menos nas linhas e mais no que deixa de depender de disciplina: isolamento, RLS, evento e auditoria não podem ser esquecidos.

Projeto completo nos dois frameworks, com multiempresa, permissões, dinheiro e eventos: Projeto de exemplo

Blog completo, do banco às views, no caso simples sem multiempresa: Exemplo 2 — Blog

Chat em tempo real, com revogação de acesso de quem está com a tela aberta: Exemplo 3 — Chat

Estratégia de adoção

O Vetra nasce como camada hospedada dentro de aplicações Rails existentes, porque ninguém reescreve um sistema em produção para trocar de framework.

1. **Pré-requisito do modo hospedado.** O app Rails hospedeiro precisa rodar em Ruby 4 e em versão do Rails compatível. Aplicações em Ruby ou Rails antigos não podem carregar o Vetra no mesmo processo.
2. **Modo hospedado (aplicações modernas).** O Vetra é montado como aplicação Rack dentro do Rails e compartilha sessão e autenticação. O Sequel reutiliza a conexão do Active Record (gem sequel-activerecord_connection), para que código Rails e comandos Vetra participem da mesma transação e do mesmo contexto de RLS. Testes obrigatórios:
   a. Active Record inicia a transação e o Sequel opera dentro dela;
   b. Sequel inicia a transação e o Active Record opera dentro dela;
   c. uma exceção desfaz as alterações dos dois ORMs;
   d. o rollback de um savepoint não dispara eventos ou callbacks indevidos;
   e. a conexão volta ao pool sem contexto de autorização da operação anterior.
3. **Modo serviço (aplicações legadas).** O Vetra roda como serviço independente, com comunicação por API e eventos via outbox. Se compartilhar o banco, cada tabela tem um único dono.
4. **Pontes.** Models Active Record existentes como fontes somente leitura; eventos do Vetra publicados para o Rails via outbox; jobs do Rails podem disparar comandos Vetra.
5. **Estrangulamento.** Domínios migram um a um, cada um com critério de saída medido.
6. **Modo independente.** Só depois de ao menos um domínio em produção real: gerador de projeto, boot e deploy próprios.

Riscos desse caminho: duas convenções no mesmo repositório (mitigação: fronteira explícita por domínio e proibição de acesso direto a models Rails fora das pontes) e, no modo serviço, duas aplicações escrevendo no mesmo banco (mitigação: dono único por tabela).

Stack e dependências

O Vetra constrói apenas o que o diferencia e reaproveita o ecossistema Ruby no resto.

Camada	Escolha	Construído pelo Vetra?
Linguagem e runtime	Ruby 4.0.x, YJIT	Não
Interface HTTP	Rack 3	Não
Servidor	Falcon (fibers); Puma como alternativa	Não
Persistência	Sequel + driver pg	Não; o Vetra adiciona repositórios e escopos
Banco	PostgreSQL 16+ (RLS, replicação lógica, SKIP LOCKED)	Não
Carregamento	Zeitwerk; Ruby::Box quando ativo	Parcial: integração com Box
Tipos	RBS gerado do grafo	Sim: geração
Grafo, comandos, políticas, capacidades	Núcleo próprio	Sim
Consultas reativas	Núcleo próprio sobre replicação lógica	Sim
Workflows e jobs	Núcleo próprio sobre PostgreSQL	Sim
Cliente frontend	SDK TypeScript para Vue e React	Sim
Observabilidade	OpenTelemetry	Não; o Vetra instrumenta
Deploy	Contêiner; Kamal compatível	Não

Versão mínima do PostgreSQL é uma questão em aberto; 16 é a hipótese de trabalho.

Registro de decisões arquiteturais (ADRs)

Vinte e uma decisões fundadoras; cada uma merece um ADR completo antes da implementação do componente correspondente. O ADR-016 é o central e deve ser escrito primeiro, porque 004 e 005 dependem dele.

ADR	Decisão	Alternativa rejeitada	Consequência principal
016	Ciclo de vida da operação: declarar, validar, autorizar, executar, persistir, auditar — com unidade de trabalho, Effect Boundary, savepoints, outbox e idempotência	Responsabilidades espalhadas entre comando, job e persistência	Contrato transacional único; cada etapa com pré-condições, garantias, limites e testes
001	Grafo compilado em IR versionada e validado no boot	Metaprogramação em runtime, estilo Rails	Verifica o que é declarado e deriva artefatos; não cobre código dinâmico arbitrário
002	Somente PostgreSQL	Suporte multibanco	RLS, replicação lógica e filas nativas; perde portabilidade
003	Sequel com camada fina de repositórios e SQL explícito	ORM próprio ou Active Record	Menos código a manter; dependência de um mantenedor central
004	Capacidades e Effect Boundary em vez de acesso global	Escopo por convenção (default scopes)	Isolamento estrutural para código no runtime; fronteira de confiança documentada

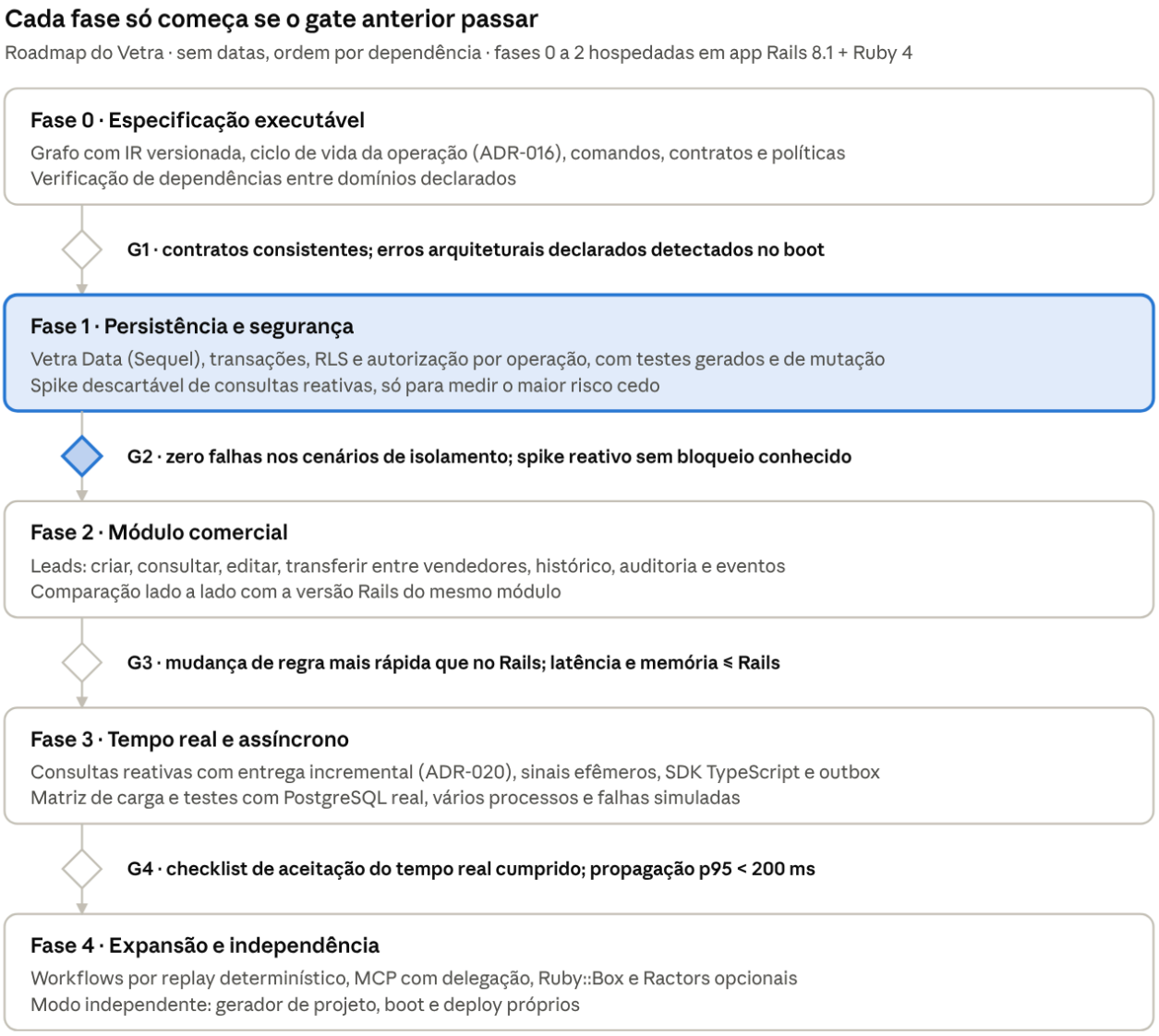
ADR	Decisão	Alternativa rejeitada	Consequência principal
005	RLS obrigatório: FORCE, papel sem BYPASSRLS, contexto assinado validado por função SECURITY DEFINER chamada como InitPlan (uma vez por consulta), resultado nunca gravado em variável de sessão, SET LOCAL na transação externa, views security_invoker, FKs compostas, negação por padrão	Isolamento só na aplicação; contexto ou sinalizador "verificado" em variável de sessão	Defesa contra erro e contra SQL não confiável; um HMAC por consulta, a medir
006	Isolamento em três níveis; Ruby::Box opcional	Ruby::Box como fundação	Núcleo independente de recurso experimental
007	Comando como unidade única de operação	Controllers + services + jobs separados	Uma regra, várias superfícies; adaptadores explicitam semântica
008	Notificação como invalidação; ordem pelo xid8 da transação, lendo só abaixo do xmin do snapshot; checkpoint durável e ressincronização	Sequência simples tratada como ordem de commit	Nenhum evento perdido por commit fora de ordem; pequeno atraso até as transações anteriores terminarem
009	Workflows por histórico de eventos e replay determinístico em Fiber	Persistir Fibers; Temporal obrigatório	Ruby linear e durável; restrições de determinismo e versionamento
010	Adoção hospedada (hosts em Ruby 4) ou modo serviço (legado)	Framework independente desde o início	Caminho real de adoção; convivência de duas convenções
011	MCP opt-in com identidade delegada, aprovação para sensíveis e limite de taxa	Agente com a identidade integral do usuário	Mais governança, menos automação irrestrita

ADR	Decisão	Alternativa rejeitada	Consequência principal
012	Canário em modo sombra com capacidades simuladas, só na fase 4	Canário executando efeitos reais	Compara versões sem efeitos duplicados; exige simulador determinístico e repositório simulado
013	Privacy, Audit e History separados; criptografia por envelope (KEK em KMS, DEK por titular em armazenamento de chaves separado); eliminar = destruir a DEK	Histórico em claro; DEKs no mesmo banco e no mesmo backup dos dados	Eliminação viável; em backups antigos de chaves, só vale após a retenção deles
014	Outbox com entrega pelo menos uma vez; handlers deduplicam pela chave do evento; efeito externo exatamente uma vez só com chave de idempotência aceita pelo provedor	Promessa de entrega exatamente uma vez	Duplicatas raras e documentadas quando o provedor não aceita idempotência
015	Migrações expand/contract com análise de locks	SQL gerado aplicado diretamente	Deploy sem parada; migrações em mais etapas
017	Tipo `money` nativo: valor + moeda, canônico na API, formato por idioma	Decimal solto e formatação manual	Sem erros de Float, moeda ou separador; um tipo a mais no núcleo
018	Camada de UI em módulos fora do núcleo (vetra-ui, -turbo, -admin, -react); Turbo como cliente; ejeção em três níveis (slots, template com props, página inteira) que substitui a apresentação, nunca a autorização; vetra doctor compara templates ejetados com o contrato	Cliente JavaScript próprio; scaffold copiado sem vínculo com o contrato	Reaproveita cliente maduro; tela ejetada continua sob comandos e políticas

ADR	Decisão	Alternativa rejeitada	Consequência principal
019	Sinais efêmeros autorizados pelo mesmo motor: "digitando" com expiração; presença por leases por usuário e dispositivo (tabela UNLOGGED no modo só PostgreSQL); LISTEN em conexão dedicada por processo; Redis opcional, adotado por métrica de latência de commit	Gravar sinais no histórico; canal sem autorização; troca por número fixo de usuários	Mesma regra de acesso dos dados; presença se perde num crash do banco, o que é aceitável
020	Entrega live incremental: revisão de autorização por trigger; modos strict (padrão) e balanced; agrupamento por chave estática + partição por evento; delta, reconsulta parcial ou completa; suspensão quando a revisão não pode ser confirmada	Reconsulta por assinante; permissões só em cache com revalidação periódica	Custo próximo ao do broadcast com revogação garantida por lote; revisão por tenant gera invalidações em massa
021	Campos de transição alteráveis só por comandos declarados (`set_by:`, `guarded:`); o CRUD gerado os omite e o grafo recusa exposições genéricas	CRUD genérico com todos os campos	Regras e eventos não podem ser contornados; um pouco mais de declaração na entidade

Roadmap e critérios de corte

O roadmap tem cinco fases sem datas: a ordem é ditada por dependência, e cada fase só começa se o gate anterior passar.



roadmap · 5 fases, 4 gates

O gate G2 é decisivo: sem isolamento sólido não há módulo comercial, e o spike reativo da fase 1 expõe cedo o maior risco técnico sem que a fundação dependa dele. As fases 0 a 2 rodam hospedadas em um app Rails 8.1 com Ruby 4; aplicações legadas usam o modo serviço.

Riscos e mitigações

O maior risco não é técnico: é a síndrome do segundo sistema — nove pilares e nenhuma entrega.

Risco	Impacto	Mitigação
Escopo grande demais	Projeto nunca entrega	Só os quatro componentes da fundação antes de qualquer extensão
Complexidade acumulada e verbosidade	Perde para o Rails em produtividade	Camadas de adoção, orçamento de conceitos e métricas de simplicidade; CRUD maior que no Rails dispara revisão do DSL
Ruby::Box experimental	Mudanças de API ou bugs no runtime	Opcional; o núcleo não depende dele
Escala das consultas reativas	Latência e carga no banco	Entrega incremental com agrupamento em dois níveis (ADR-020); matriz de carga; medir o custo do NOTIFY no commit
Revogação com assinatura aberta	Dados entregues após perda de acesso	Revisão de autorização por trigger; modo strict confere a revisão antes de cada lote; suspensão quando a revisão não pode ser confirmada
Grupo de renderização misturando permissões	Vazamento entre usuários	Chave inclui tenant e escopo efetivo; partição por predicado; teste de aceitação de isolamento
Eventos fora da ordem de commit	Assinante perde uma alteração	xid8 + leitura abaixo do xmin do snapshot; checkpoint e ressincronização
CRUD contornando regras de negócio	Transição sem evento ou sem campos derivados	Campos de transição (ADR-021)
Efeitos duplicados no canário	Escritas e envios em dobro	Modo sombra com capacidades simuladas
Entrega repetida pela outbox	Notificações ou cobranças duplicadas	Deduplicação pela chave do evento; chave de idempotência no provedor externo quando ele aceitar

Risco	Impacto	Mitigação
Restauração de backup após eliminação	Dados de titular eliminados voltam a ser legíveis	DEKs em armazenamento de chaves separado, com retenção curta; prazo documentado
Cache com permissões	Mistura de contextos ou acesso revogado	Chaves por escopo; invalidação por autorização
Migrações geradas	Locks e indisponibilidade	Expand/contract e análise de locks
Versionamento de workflows	Execuções corrompidas após deploy	Versão por workflow; execuções antigas terminam na versão original
Gems com conexão própria	Contornam as capacidades	Contexto assinado com negação por padrão; classificação de dependências
Host Rails legado	Modo hospedado inviável	Modo serviço via API e eventos
Extensões C não seguras para Ractor	Falhas no pool de CPU	Lista de bibliotecas aprovadas; fallback para processos
Gems que assumem Rails	Ecossistema menor na prática	Modo hospedado reaproveita gems do Rails hospedeiro
Custo de RLS em consultas complexas	Planos de execução piores	Função de contexto como InitPlan; benchmarks com RLS ativo desde a fase 1
Dependência de Sequel	Ritmo de manutenção externo	Camada fina de repositórios isola a dependência

Métricas de sucesso

A métrica principal é o tempo para implementar uma mudança de regra de negócio com testes e segurança preservados; linhas de código são apenas observação. Todas as medições comparam o mesmo módulo comercial em Rails 8.1 e em Vetra, na mesma infraestrutura, e as metas são hipóteses a calibrar no protótipo.

Métrica	Meta proposta	Tipo
Tempo para implementar uma mudança de regra (equipe externa, com testes)	Menor que no Rails	Principal
Cenários de isolamento: gerados, mutação e adversariais	Zero falhas; toda mutação detectada	Eliminatória (não é prova formal)
Latência de API (p95), mesmo endpoint	⩽ Rails	Eliminatória
Propagação de mudança até a tela (p95)	< 200 ms	Eliminatória da fase de tempo real
Revogação no modo strict	Nenhum lote iniciado após o commit da revogação entrega dados	Eliminatória da fase de tempo real
Revogação no modo balanced	Efetiva em até um intervalo de revalidação	Eliminatória da fase de tempo real
Reconexão após eventos perdidos	Estado autorizado recomposto	Eliminatória da fase de tempo real
Código do CRUD multiempresa	Não maior que no Rails sem isolamento	Alerta de verbosidade
Tempo de um dev Rails até o primeiro CRUD multiempresa	⩽ 1 dia	Simplicidade
Conceitos necessários na camada 1	⩽ 3	Simplicidade
Arquivos escritos à mão para um CRUD	1 a 2	Simplicidade
Memória por processo sob carga	⩽ Rails	Diferenciação
Custo do contexto assinado	Latência com e sem HMAC	Observação
Linhas do módulo completo	Medir	Observação
Conexões simultâneas por processo (SSE)	Medir	Observação

Métrica eliminatória que falhar → o pilar envolvido é cortado ou redesenhado antes da fase seguinte.

Matriz de carga do tempo real

São cenários para benchmark, não limites conhecidos do PostgreSQL ou do Redis. Cada cenário roda duas vezes — com todos os usuários no mesmo grupo de permissão e com permissões individuais —, e alterações persistidas e sinais efêmeros são medidos separadamente.

Cenário	Conexões simultâneas	Eventos por segundo
Pequeno	50	1
Médio	200	10
Grande	1.000	50
Estresse	5.000	100

Medidas: latência p50, p95 e p99; CPU e memória; consultas por alteração; conexões ao banco; tempo até a tela; tempo de propagação da revogação; impacto do NOTIFY na latência de commit. O Redis passa a ser recomendado quando o NOTIFY começar a aumentar a latência de commit das transações de negócio.

Critérios de aceitação do tempo real (gate G4)

- [ ] Usuário removido da conversa deixa de receber novas entregas após a revogação, conforme o modo
- [ ] Falha ou atraso no barramento não permite uso indefinido de permissões antigas
- [ ] Usuários de empresas diferentes nunca compartilham grupo de renderização
- [ ] Nova mensagem não exige reexecutar a consulta completa de todos os assinantes
- [ ] Nova mensagem num chat gera no máximo uma renderização por combinação de predicados (autor e demais)
- [ ] Mudança de filtro, visibilidade ou ordenação produz atualização correta
- [ ] Reconexão após eventos perdidos recompõe o estado autorizado
- [ ] Indicador de digitação expira após queda inesperada da conexão
- [ ] Presença continua correta quando um de vários dispositivos desconecta
- [ ] Tela ejetada continua usando os mesmos comandos e regras de autorização
- [ ] Execução correta com vários processos e falhas simuladas, usando PostgreSQL real e os papéis de banco de produção

Questões em aberto

- [ ] O nome "Vetra" está disponível no RubyGems, no GitHub e como marca?

- [ ] Qual a versão mínima do PostgreSQL (16 ou 17)? `security_invoker` exige 15+; `xid8` e `pg_current_snapshot`, 13+.
- [ ] Qual o custo medido do HMAC por consulta no RLS, e como funcionam emissão, validade e rotação da chave do contexto assinado?
- [ ] Qual o esquema da IR do grafo e sua política de compatibilidade entre versões?
- [ ] Como modelar escopos com vínculos múltiplos (usuário em várias unidades e empresas) sem custo de avaliação proibitivo?
- [ ] O modo hospedado aceita escrita em tabelas do Rails, ou só leitura nas pontes?
- [ ] Qual o modelo de licença e governança do projeto?
- [ ] Qual o comportamento do Ruby::Box com as gems nativas mais usadas (pg, nokogiri)?
- [ ] A revisão de autorização por tenant gera invalidações em massa demais? A partir de quando vale granularidade por recurso (sala, revenda)?
- [ ] Como permissões vindas de fora do banco (SSO, provedor de identidade) geram revisão, já que não disparam trigger?
- [ ] Qual o intervalo do modo balanced e o tempo máximo antes de suspender a entrega por perda de sincronia?
- [ ] Quais formas de consulta aceitam delta direto e quais caem em reconsulta parcial ou completa?
- [ ] Vale implementar ordem estrita entre revogação e entrega em vários processos, ou a janela de milissegundos do modo strict é aceitável?
- [ ] Em que volume o NOTIFY passa a aumentar a latência de commit (matriz de carga)?
- [ ] Onde fica o armazenamento de chaves do crypto-shredding (KMS, banco separado), qual a retenção dos backups dele e como é feita a rotação da KEK?
- [ ] ADR-018: como o gerador versiona templates ejetados, e o que o `vetra doctor` consegue verificar em ERB dinâmico?
- [ ] Quem classifica comandos como sensíveis para MCP, e qual o padrão por tipo de operação?

Resumo das pendências, separadas entre decisões, medições e itens estruturais:
Pendências (rodada 7)

Fontes

- Ruby 4.0.0 Released — ruby-lang.org
- Ruby 4.0.0 リリース — ruby-lang.org
- Ruby 4.0.9 portable binaries — GitHub
- Temporal Ruby — crash-proof fibers — Temporal
- System Information Functions (snapshot e xid8) — PostgreSQL

- Features (modos de pooling) — PgBouncer
- RLS Performance and Best Practices — Supabase

Dados sobre o Rails 8.1 são de conhecimento prévio, sem verificação nesta data.